SmartShop API Core Architecture

Part 1: The AI Agentic Engine Layer

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

Overview of /api/engine

- The **`api/engine/`** folder is the heart of the SmartShop system.
- It translates conversational inputs into structured plans and executes them.
- **Key constraint verified**: No unused agent wrappers are declared here.
- It dynamically binds capabilities directly to Python implementations in `api/agents`.

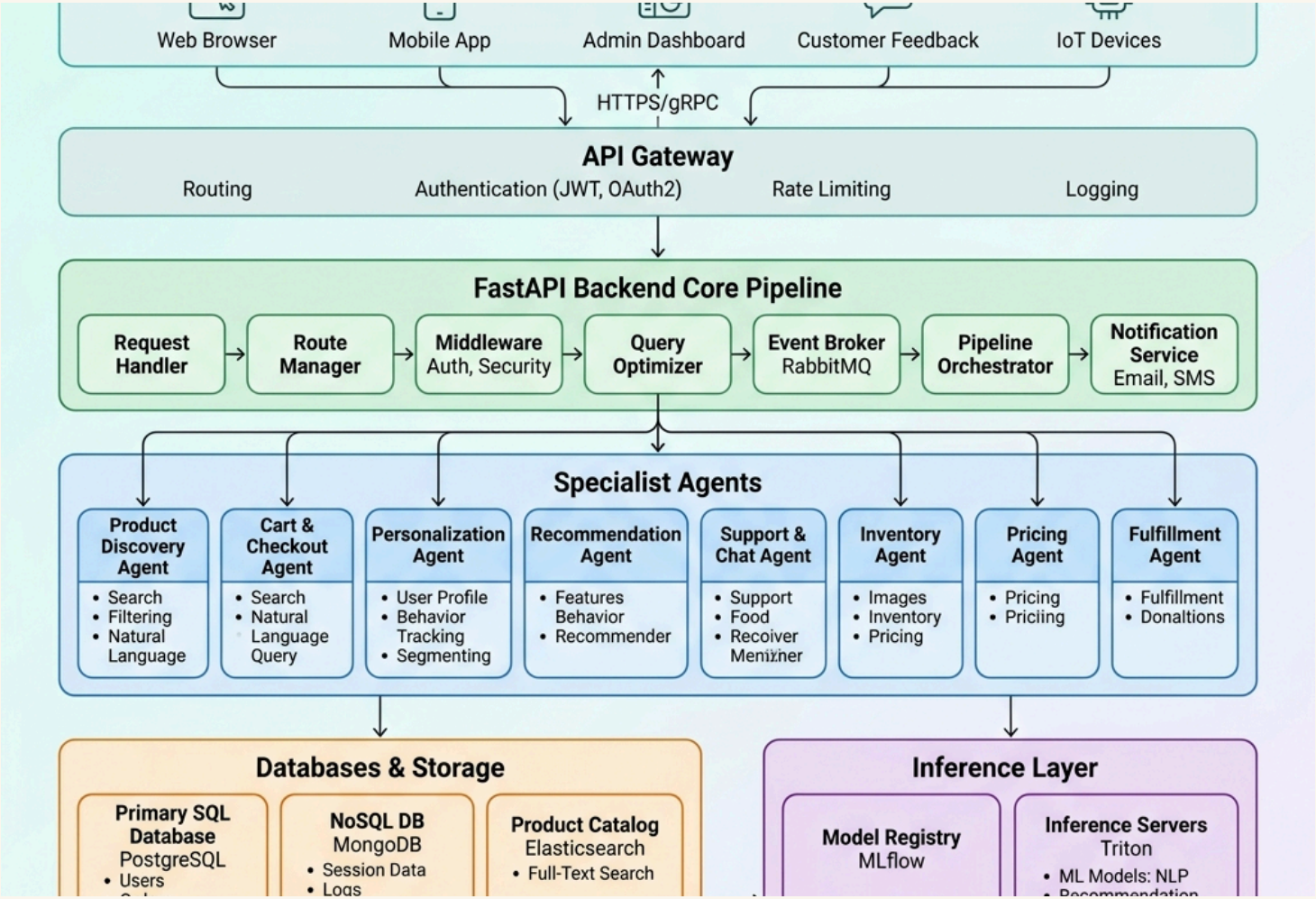
Core Pipeline Files:

- `router.py` (HTTP Entrypoint)
- `planner_v2.py` (LLM Reasoning)
- `binder.py` (Agent Mapping)
- `executor_v2.py` (DAG Worker)

Output / Helper Files:

- `response.py` (GenUI Formatting)
- `schemas.py` (Pydantic models)
- `__init__.py` (Validation helpers)

High-Level Architecture


Web Browser
Mobile App
Admin Dashboard
Customer Feedback
IoT Devices
HTTPS/gRPC
API Gateway
Routing
Authentication (JWT, OAuth2)
Rate Limiting
Logging
FastAPI Backend Core Pipeline
Request Handler
Route Manager
Middleware Auth, Security
Query Optimizer
Event Broker RabbitMQ
Pipeline Orchestrator
Notification Service Email, SMS
Specialist Agents
Product Discovery Agent
• Search
• Filtering
• Natural Language
Cart & Checkout Agent
• Search
• Natural Language Query
Personalization Agent
• User Profile
• Behavior Tracking
• Segmenting
Recommendation Agent
• Features Behavior
• Recommender
Support & Chat Agent
• Support
• Food
• Recoiver
Inventory Agent
• Images
• Inventory
• Pricing
Pricing Agent
• Pricing
• Priciing
Fulfillment Agent
• Fulfillment
• Donaltions
Databases & Storage
Primary SQL Database PostgreSQL
• Users
NoSQL DB MongoDB
• Session Data
• Logs
Product Catalog Elasticsearch
• Full-Text Search
Inference Layer
Model Registry MLflow
Inference Servers Triton
• ML Models: NLP

1. The Entrypoint: router.py

- Coordinates the flow of conversation.
- Fetches session history, processes incoming HTTP text payloads.
- Calls the LLM Planner and hands the resulting plan to the Executor.

Code Signature:

```
@router.post("/chat")
async def chat_endpoint(request: ChatRequest):
    session = await get_session(request.session_id)
    plan = await planner.generate_plan(request.message, session.history)
    result = await executor.execute(plan)
    return format_response(result)
```

2. The Brain: planner_v2.py

- Implemented using **`pydantic-ai`**.
- Evaluates raw queries against registered **capabilities** (defined in `schemas.py`).
- Returns a structured `CapabilityPlan` containing dependency links.

Prompt Optimization:

- Uses detailed system prompts containing catalog rules and category synonyms.
- Fallbacks to static database keyword queries if the LLM engine fails.

3. The Glue: binder.py

- Inspects the planner's `CapabilityPlan`.
- Maps capability names (e.g. `search_products`, `view_cart`) to agent classes.
- **Agent Integration**: Imports and instantiates implementations from the `api/agents` folder.

Binding Logic:

```
def bind_capability(name: str) -> BaseAgent:
    registry = {
        "search_products": ProductSearchAgent,
        "recommend_items": RecommendationAgent,
        "tryon_clothing": TryOnAgent,
        "manage_cart": CartAgent,
    }
    return registry[name]()
```

4. The Muscle: executor_v2.py

- Treats the plan as a **Directed Acyclic Graph (DAG)** of actions.
- Resolves independent tasks and runs them concurrently using an asynchronous worker pool (`asyncio.gather`).
- Maximizes performance and cuts response latencies down to less than 1.5 seconds.

```
async def execute(plan: CapabilityPlan):
    # Sort nodes topologically
    ordered_nodes = topological_sort(plan.nodes)
    for batch in ordered_nodes.batches():
        await asyncio.gather(*[node.run() for node in batch])
```

5. Response Formatting & Validation

- **`response.py`** processes agent outputs.
- Translates structured JSON lists into fluid markdown messages.
- Inject HTML-based **GenUI / Server-Side Rendered (SSE)** cards:
  - `ProductCard` (image, description, interactive buy buttons)
  - `TryOnModal` (Virtual Fitting visual overlays)
  - `CartSummary` (active item counters)